

PORTADA

Arquitectura de Software

Semana 1 — Introducción a los ambientes Backend, Frontend y Fullstack

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

6 personas
250.000 visitantes



HealthCare.gov, en cifras

Sistema	HealthCare.gov — portal federal de compra de seguros de salud (EE. UU.)
Lanzamiento	1 de octubre de 2013
Visitantes día 1	~250.000 personas intentando registrarse
Inscripciones exitosas día 1	Solo 6
Sistemas externos a integrar	Más de 50 (IRS, Seguridad Social, aseguradoras, verificación migratoria)
Costo acumulado del proyecto	Más de 2.100 millones de dólares (contratistas, desde 2011)
Desenlace	Equipo de rescate tecnológico reconstruye la plataforma en ~2 meses

Cifras reportadas por la Oficina de Rendición de Cuentas de EE. UU. (GAO) y cobertura de prensa contemporánea.

CASO · LO QUE SE VEÍA

El frontend parecía estar bien

La página de inicio y el formulario de registro se veían pulidos: campos claros, diseño moderno, sin errores visibles de diagramación. **Cualquiera que abriera el sitio pensaría que el sistema funcionaba.**

El problema no era lo que el usuario veía en la pantalla — era **lo que pasaba después de dar clic en "enviar"**, en servidores que el usuario nunca ve directamente.



CASO · LO QUE FALLABA

Detrás de la pantalla: el backend

Cada solicitud de registro debía pasar por un componente central (el "**data hub**") que consultaba en tiempo real más de **50 sistemas externos** — Hacienda, Seguridad Social, aseguradoras privadas — para verificar identidad, ingresos y elegibilidad.

- El data hub no había sido probado bajo la carga real de lanzamiento.
- Los servidores no escalaban: cada usuario adicional multiplicaba el tiempo de espera de todos.
- Errores de un sistema externo dejaban la sesión del usuario colgada, sin mensaje claro.

El código visible (frontend) apenas era el 5 % del problema real.

TRANSICIÓN

Dos sistemas, no uno

HealthCare.gov no era "un sitio web que falló". Era **dos sistemas conectados** — uno que el usuario ve y toca, y otro que procesa, calcula y guarda — y solo el segundo colapsó.

Para hablar con precisión de qué falló, primero hay que nombrar bien cada parte. Eso es lo que hace esta sesión.

CONCEPTO

Frontend: la parte que se ve y se toca

Frontend (*front-end*, "*extremo delantero*") es la porción de un sistema que se ejecuta en el dispositivo del usuario y con la que este interactúa directamente: navegador web, app móvil, aplicación de escritorio.

- Renderiza la interfaz: botones, formularios, texto, imágenes.
- Captura las acciones del usuario (clics, teclas, gestos).
- Envía y recibe datos del backend, normalmente por HTTP.

Tecnologías típicas: HTML, CSS, JavaScript y frameworks como React, Angular o Vue.



CONCEPTO

Backend: la parte que procesa y decide

Backend (*back-end*, "extremo trasero") es la porción de un sistema que se ejecuta en un servidor, fuera del alcance directo del usuario: recibe solicitudes, aplica reglas de negocio, consulta o guarda datos, y responde.

- No tiene interfaz visual propia — expone datos y funciones a través de una API.
- Aplica validaciones, cálculos y reglas de negocio que no deben vivir en el navegador.
- Se conecta a bases de datos y a otros sistemas (como los 50+ de HealthCare.gov).

Tecnologías típicas: Java/Spring, Node.js, Python/Django, .NET, junto con una base de datos.



CONTRA EJEMPLO

Mito: "el frontend es solo diseño"

Es común pensar que el frontend es "la parte bonita" y el backend "la parte que piensa". **Es una simplificación engañosa.**

- El frontend también valida datos, gestiona estado de la aplicación y toma decisiones (qué mostrar, cuándo deshabilitar un botón).
- El backend también decide "qué se ve": qué datos exponer, en qué orden, con qué permisos.

La diferencia real no es "diseño vs. lógica" — es dónde se ejecuta el código: en el dispositivo del usuario, o en un servidor.

CONCEPTO

Cómo se conectan: el protocolo HTTP

Frontend y backend son programas independientes que se comunican por red, casi siempre usando **HTTP** (*HyperText Transfer Protocol*): el frontend envía una **solicitud** (request) y el backend responde con una **respuesta** (response), típicamente en formato **JSON**.

Elemento	Ejemplo
Método	GET , POST , PUT , DELETE
Ruta	/api/solicitudes/{id}
Cuerpo (body)	{"nombre": "Ana", "ingreso": 32000}

En HealthCare.gov, cada solicitud del navegador desencadenaba, del otro lado, docenas de llamadas HTTP internas a sistemas externos.

CONCEPTO

Fullstack: dominar ambos extremos

Un desarrollador o desarrolladora **fullstack** es quien puede trabajar tanto en el frontend como en el backend de una aplicación: construir la interfaz, y también la API y la lógica de servidor que la alimenta.

- No implica ser experto al mismo nivel en ambos lados — implica poder moverse entre ellos con criterio.
- Es especialmente valioso en equipos pequeños o en fases tempranas de un proyecto (como el Sprint 1 de este curso).

"Fullstack" describe una persona o un rol; no es una tercera capa técnica del sistema.

CONTRAEJEMPLO

Mito: "fullstack = experto en todo"

Nadie domina a fondo simultáneamente diseño de interfaces, frameworks de frontend, arquitectura de backend, bases de datos, infraestructura y seguridad. **Ese nivel de dominio total no es lo que describe el término.**

Fullstack describe amplitud de competencia suficiente para construir un flujo completo — de la pantalla a la base de datos y de vuelta — no profundidad experta en cada capa por separado.

CONCEPTO

Los tres, comparados

	Frontend	Backend	Fullstack
Dónde corre	Dispositivo del usuario	Servidor	Ambos
Qué produce	Interfaz, interacción	Datos, reglas, cálculos	El flujo completo
Lenguajes típicos	HTML, CSS, JavaScript	Java, Python, Node.js, .NET	Combinación de ambos
El usuario lo ve	Directamente	Nunca directamente	—

Ningún desarrollo real es "solo frontend" o "solo backend" en aislamiento: siempre hay una conexión entre los dos.

CONCEPTO · APLICADO AL CASO

Ubicando HealthCare.gov en el mapa

Frontend	Formulario de registro, validaciones visuales, mensajes de error en pantalla — funcionaba razonablemente bien
Backend	Data hub, verificación de identidad e ingresos, cálculo de subsidios contra 50+ sistemas externos — aquí ocurrió el colapso
Fullstack	El equipo de rescate de 2013 necesitó personas capaces de rediseñar la comunicación entre ambos extremos, no solo arreglar la pantalla

Un frontend correcto no compensa un backend que no escala.

TENSIÓN CONCEPTUAL

¿Un buen frontend garantiza un buen sistema?

Para el usuario que abre HealthCare.gov el 1 de octubre de 2013, el sistema "se veía bien".

- Desde la experiencia visual: parecía un sitio profesional y confiable.
- Desde la arquitectura: era un sistema que no podía sostener su propia carga.

¿Cuál de las dos miradas debería pesar más al decidir si un sistema está "listo para producción"?

INTERACCIÓN

Clasifiquen los componentes

Piensen en una app que usen a diario (banco, red social, plataforma de streaming). Clasifiquen cada elemento como Frontend, Backend o ambigüo:

1. El botón de "me gusta" que cambia de color al presionarlo.
2. El cálculo de cuántas personas dieron "me gusta" a una publicación.
3. El mensaje "sin conexión a internet" que aparece sin señal.
4. La validación de que una contraseña tenga mínimo 8 caracteres.

5 minutos · respondan en voz alta o en el chat

CONCEPTO · PUENTE A LA PRÓXIMA SESIÓN

Lo que todavía no hemos nombrado

Ya sabemos distinguir dónde corre el código y qué hace cada extremo. Pero HealthCare.gov no falló por "tener backend" — falló por **cómo estaban organizadas y conectadas las piezas de ese backend**: qué tan acoplado estaba el data hub, si había forma de aislar un fallo, si el diseño se probó bajo carga real antes de lanzar.

A ese conjunto de decisiones estructurales —no al código línea por línea— lo llamamos *arquitectura de software*, y es el tema de la próxima sesión.

SÍNTESIS

Ideas clave de hoy

1. **Frontend** corre en el dispositivo del usuario; **backend** corre en un servidor y nunca se ve directamente.
2. Frontend y backend se comunican por red, típicamente con **HTTP** y datos en **JSON**.
3. **Fullstack** describe a una persona capaz de moverse entre ambos extremos, no una tercera capa técnica.
4. Un frontend que "se ve bien" no garantiza que el sistema funcione: HealthCare.gov lo demostró con 250.000 visitantes y solo 6 inscripciones el primer día.
5. Lo que faltó en ese backend no fue código — fue **arquitectura**: la estructura de sus piezas y sus conexiones.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

No es un ejercicio para entregar: el docente construye en vivo un flujo mínimo **frontend + backend** y explica, paso a paso, dónde vive cada pieza y por qué.

- **Backend:** un endpoint REST en **Java Spring Boot** que expone datos en JSON.
- **Frontend:** una página en **HTML + JavaScript simple** que consume ese endpoint con `fetch()` y pinta el resultado en pantalla.

INSTRUCCIONES · BACKEND

El endpoint en Spring Boot

```
@RestController
@RequestMapping("/api/saludo")
public class SaludoController {

    @GetMapping
    public Map<String, String> obtenerSaludo() {
        Map<String, String> respuesta = new HashMap<>();
        respuesta.put("mensaje", "Hola desde el backend");
        respuesta.put("origen", "Spring Boot");
        return respuesta; // Spring lo serializa a JSON
    }
}
```

Este método corre en el servidor. El navegador nunca ve este código — solo el JSON que produce.

INSTRUCCIONES · FRONTEND

El consumo desde HTML + JavaScript

```
<p id="salida">Cargando...</p>
<script>
  fetch("http://localhost:8080/api/saludo")
    .then(resp => resp.json())
    .then(datos => {
      document.getElementById("salida").textContent =
        datos.mensaje + " (" + datos.origen + ")";
    });
</script>
```

Este código corre en el navegador del usuario — hace la solicitud HTTP, recibe el JSON y actualiza la pantalla. Ninguna de las dos partes puede mostrar el resultado completo sin la otra.

Entregable: ninguno para esta sesión — el objetivo es observar la demo y poder explicar qué línea corre dónde.

CIERRE

Para la próxima sesión

- Repasar la diferencia entre frontend, backend y fullstack con sus propias palabras — se retoma al inicio del jueves.
- Pensar en una decisión de diseño que hayan tomado en un proyecto propio y que fuera **difícil de cambiar después** — se usará como ejemplo en clase.

El jueves entramos a la pregunta central del curso: ¿qué es —y qué no es— arquitectura de software?

